Software Engineering

Lecture 3

What is modeling?

- Modeling consists of building an abstraction of reality.
- Abstractions are simplifications because:
  - They ignore irrelevant details and
  - They only represent the relevant details.
- What is *relevant* or *irrelevant* depends on the purpose of the model.

Why model software?

Why model software?

- Software is getting increasingly more complex
  - Windows XP > 40 mio lines of code
  - A single programmer cannot manage this amount of code in its entirety.
- Code is not easily understandable by developers who did not write it
- We need simpler representations for complex systems
  - Modeling is a mean for dealing with complexity

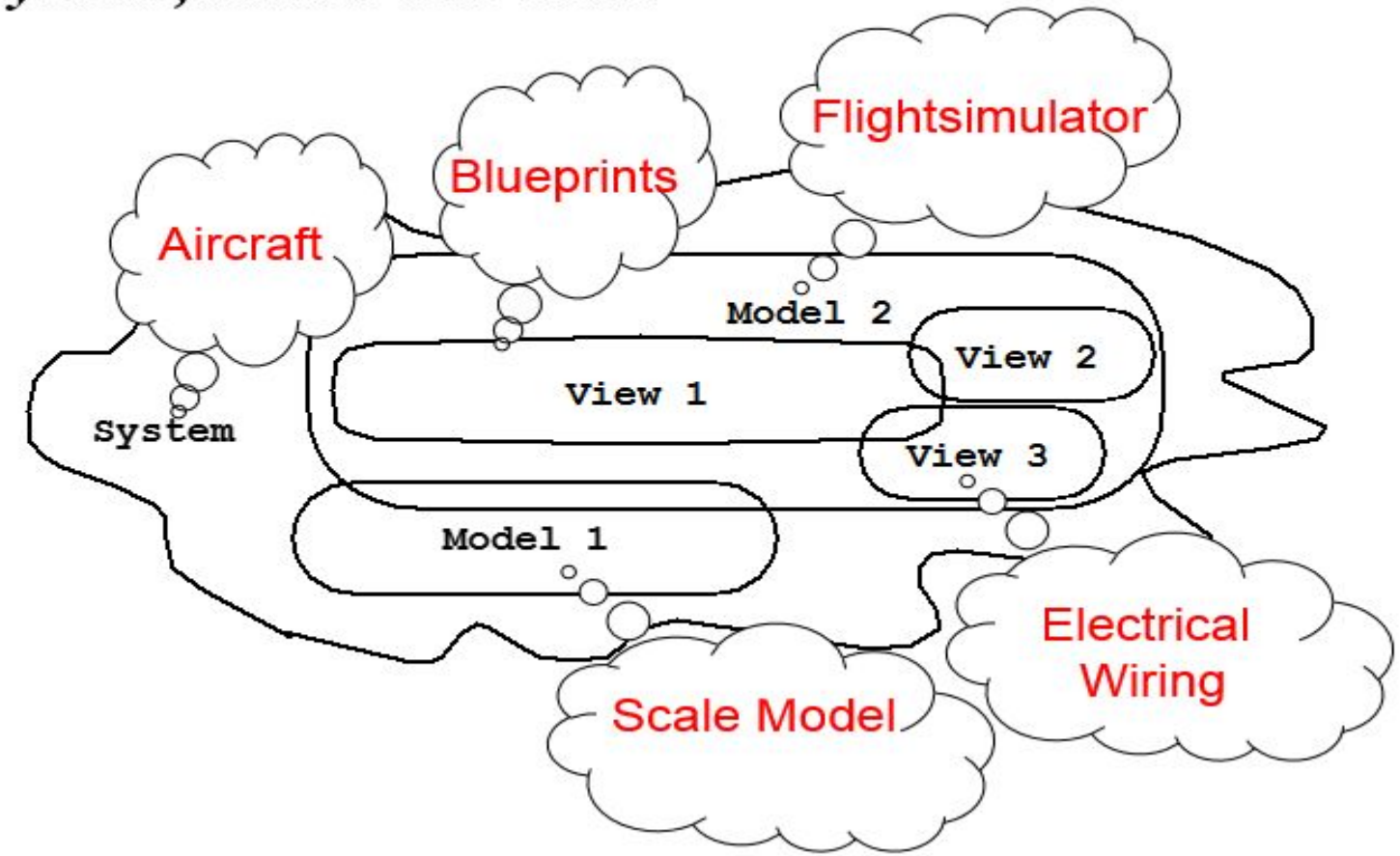
Systems, Models and Views

- A ***model*** is an abstraction describing a subset of a system
- A ***view*** depicts selected aspects of a model
- A ***notation*** is a set of graphical or textual rules for depicting views
- Views and models of a single system may overlap each other

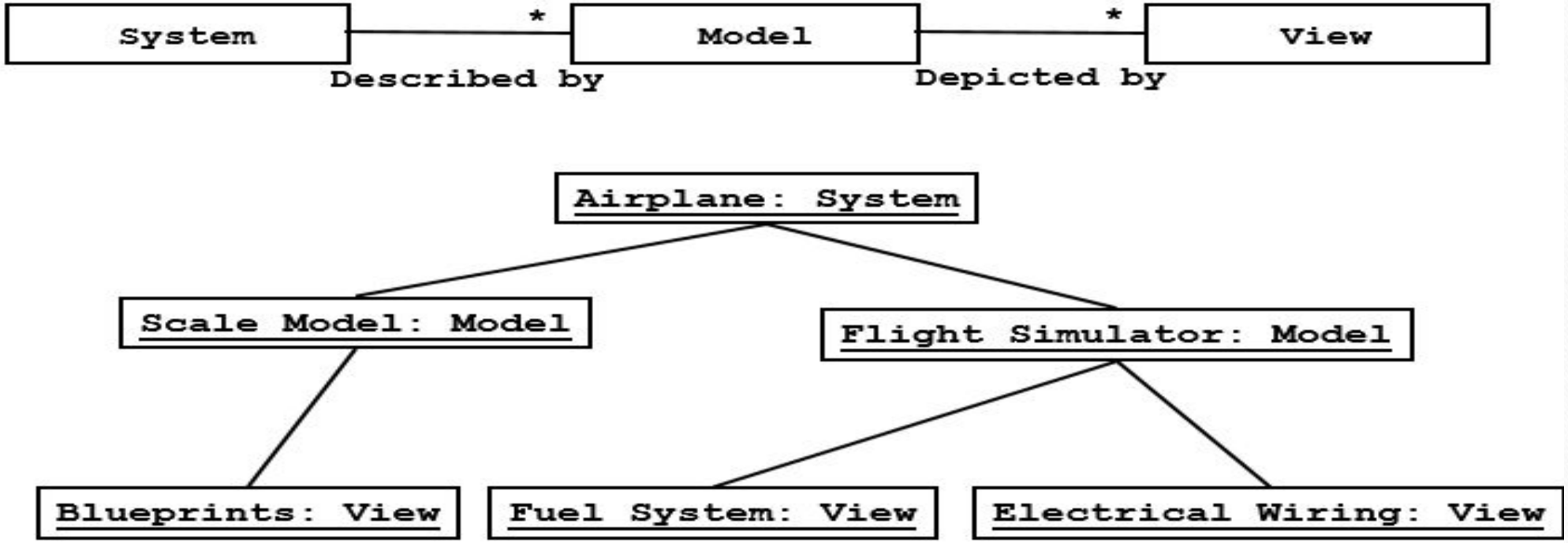
Examples:

- System: Aircraft
- Models: Flight simulator, scale model
- Views: All blueprints, electrical wiring, fuel system

Systems, Models and Views



Models, Views and Systems (UML)

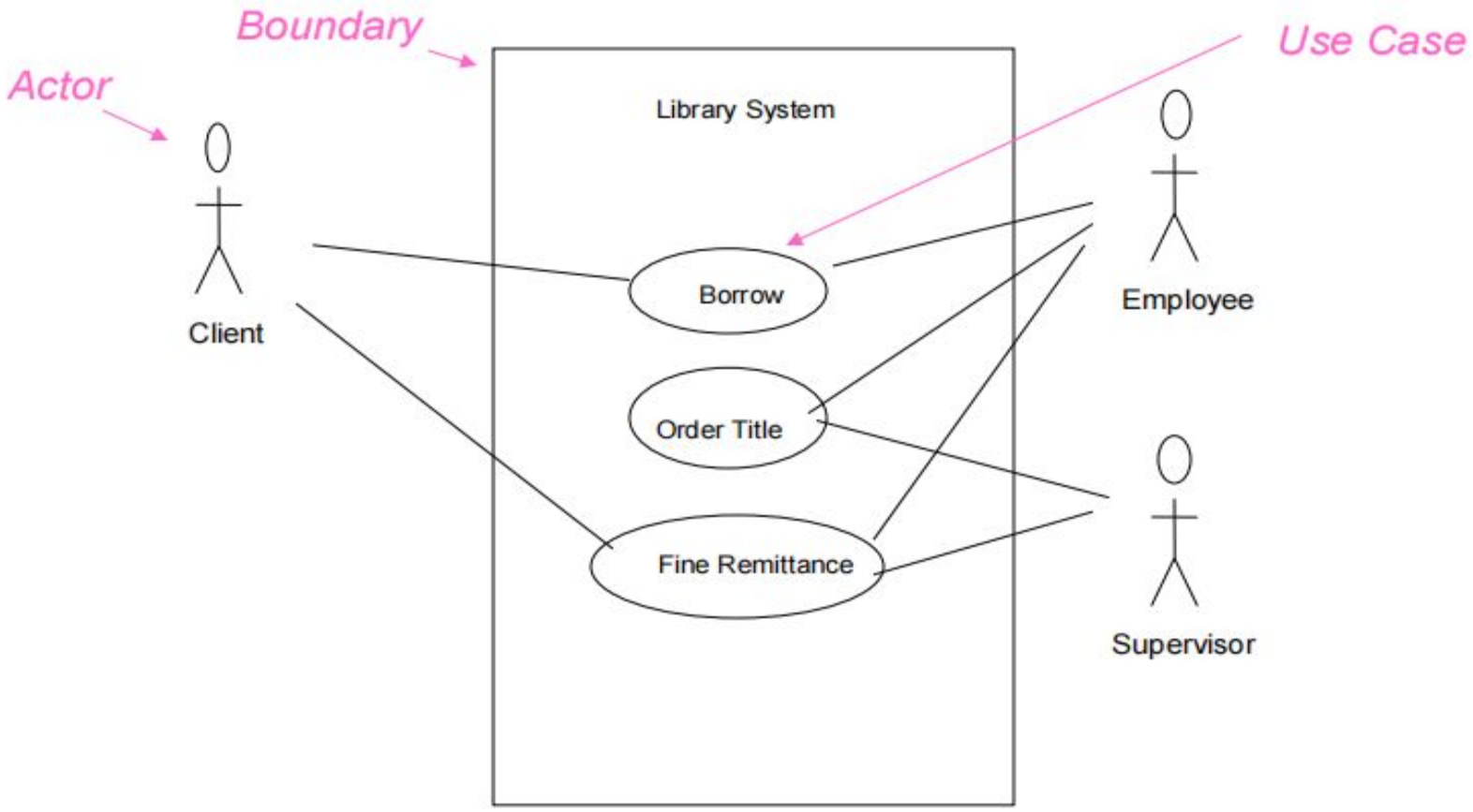


What is UML?

- UML (Unified Modeling Language)
  - An emerging standard for modeling object-oriented software.
  - Resulted from the convergence of notations from three leading object-oriented methods:
    - OMT (James Rumbaugh)
    - OOSE (Ivar Jacobson)
    - Booch (Grady Booch)
- Reference: "The Unified Modeling Language User Guide", Addison Wesley, 1999.
- Supported by several CASE tools
  - Rational ROSE
  - TogetherJ

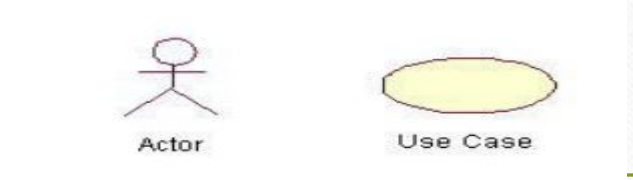
Types of UML Diagrams:

- **Use case Diagrams**
  - Describe the functional behavior of the system as seen by the user.
- **Class diagrams**
  - Describe the static structure of the system: Objects, Attributes, Associations
- **Sequence diagrams**
  - Describe the dynamic behavior between actors and the system and between objects of the system
- **Statechart diagrams**
  - Describe the dynamic behavior of an individual object (essentially a finite state automaton)
- **Activity Diagrams**
  - Model the dynamic behavior of a system, in particular the workflow (essentially a flowchart)


Boundary
Use Case
Actor
Library System
Client
Borrow
Employee
Order Title
Fine Remittance
Supervisor

Use-Case Diagrams

- **Actors**: A role that a user plays with respect to the system, including human
- users and other systems. e.g., inanimate physical objects (e.g. robot); an
- external system that needs some information from the current system.
- **Use case**: A set of scenarios that describing an interaction between a user
- and a system, including alternatives.
- **System boundary**: rectangle diagram representing the boundary between
- the actors and the system.



- Association:

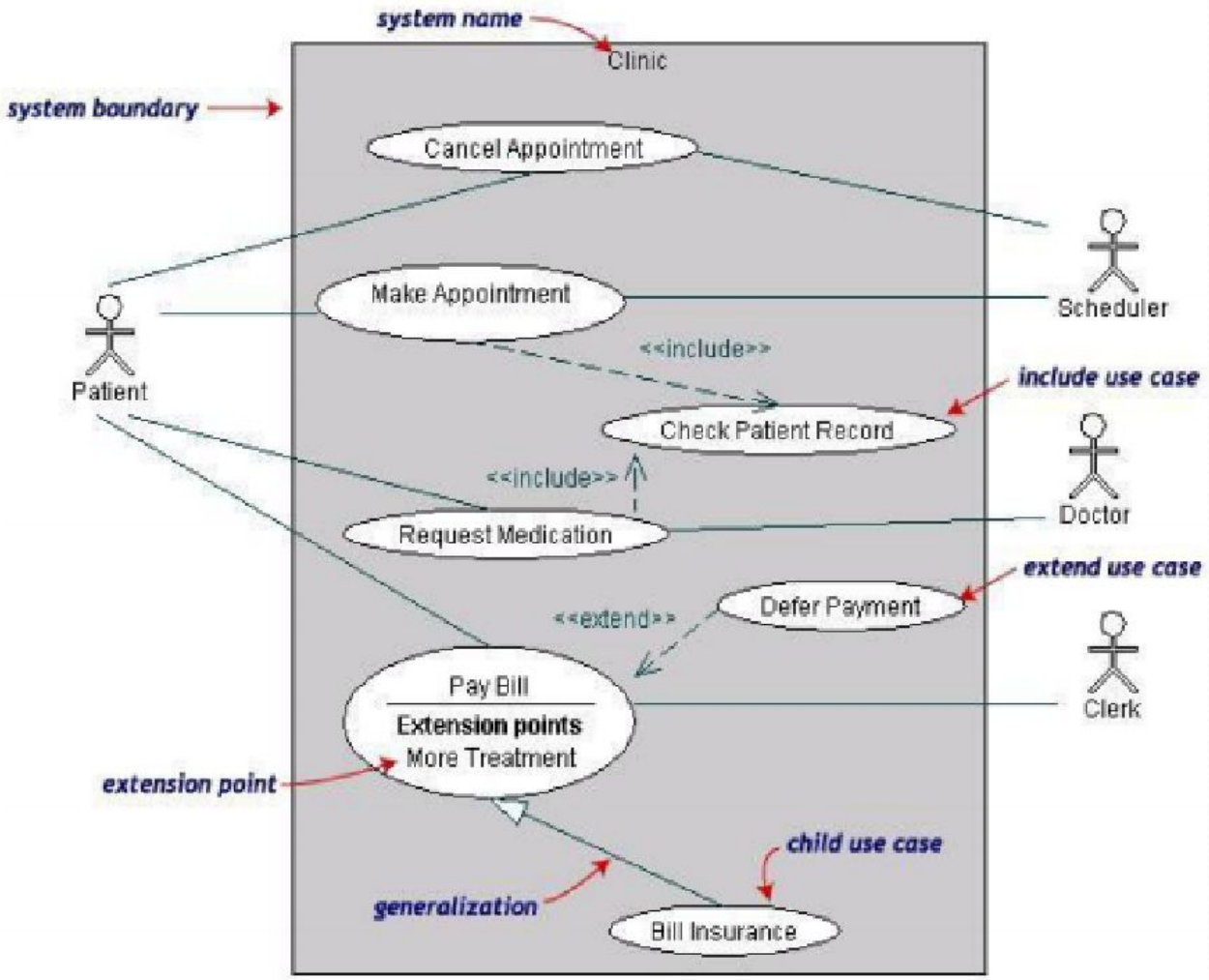
communication between an actor and a use case; Represented by a solid line.

- Generalization: relationship between one general use case and a special use case (used for defining special alternatives) Represented by a line with a triangular arrow head toward the parent use case.

- Both **Make Appointment** and **Request Medication** include **Check Patient Record** as a subtask (include)

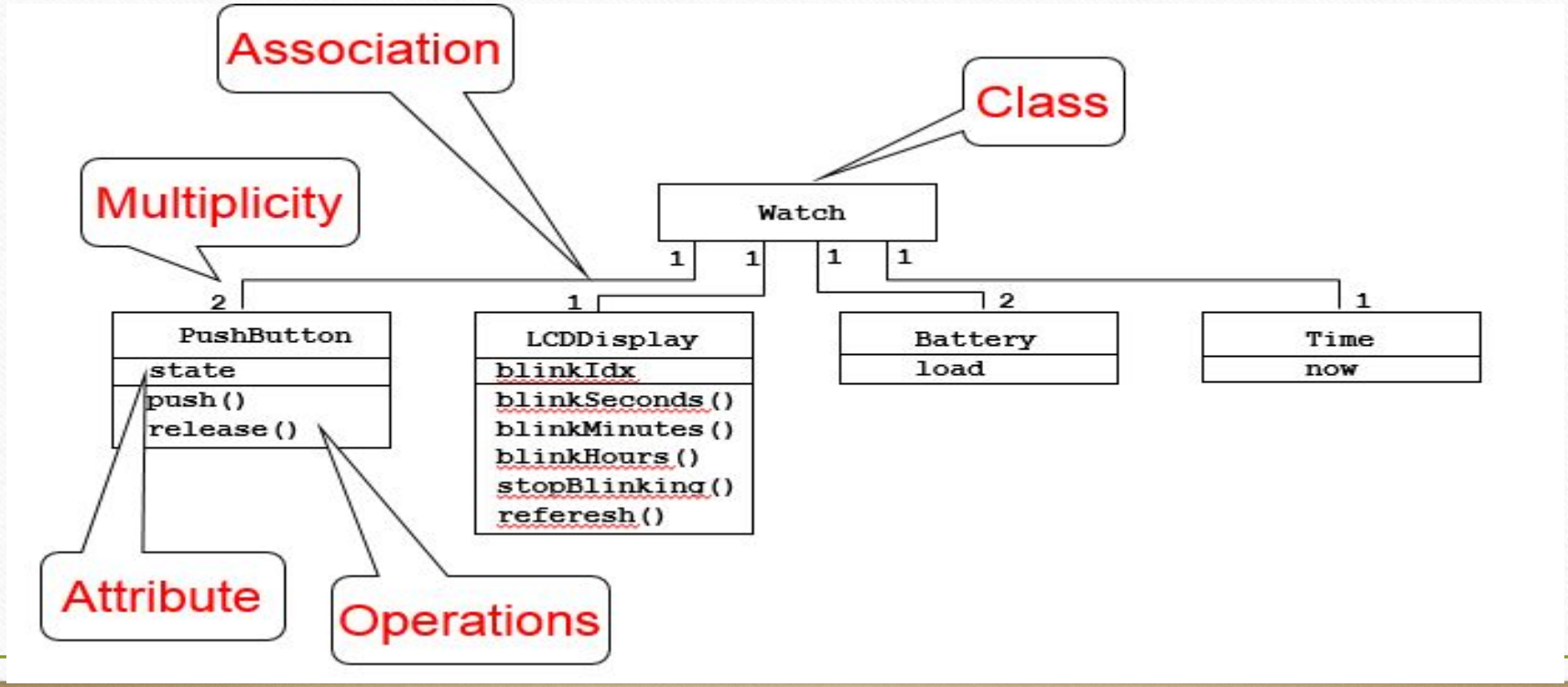
- The **extension point** is written inside the base case **Pay bill**; the extending class **Defer payment** adds the behavior of this extension point. (extend)

- **Pay Bill** is a parent use case and **Bill Insurance** is the child use case. (generalization)

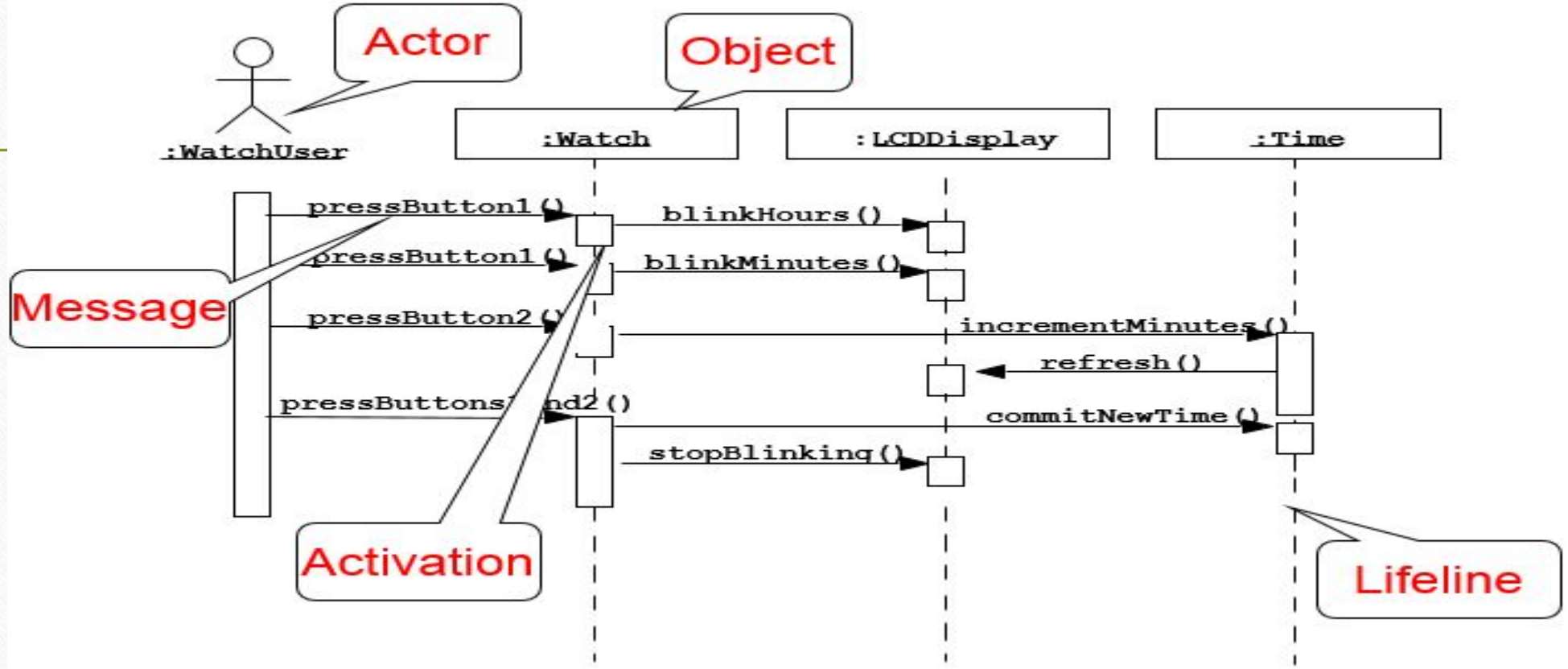


UML first pass: Class diagrams

Class diagrams represent the structure of the system

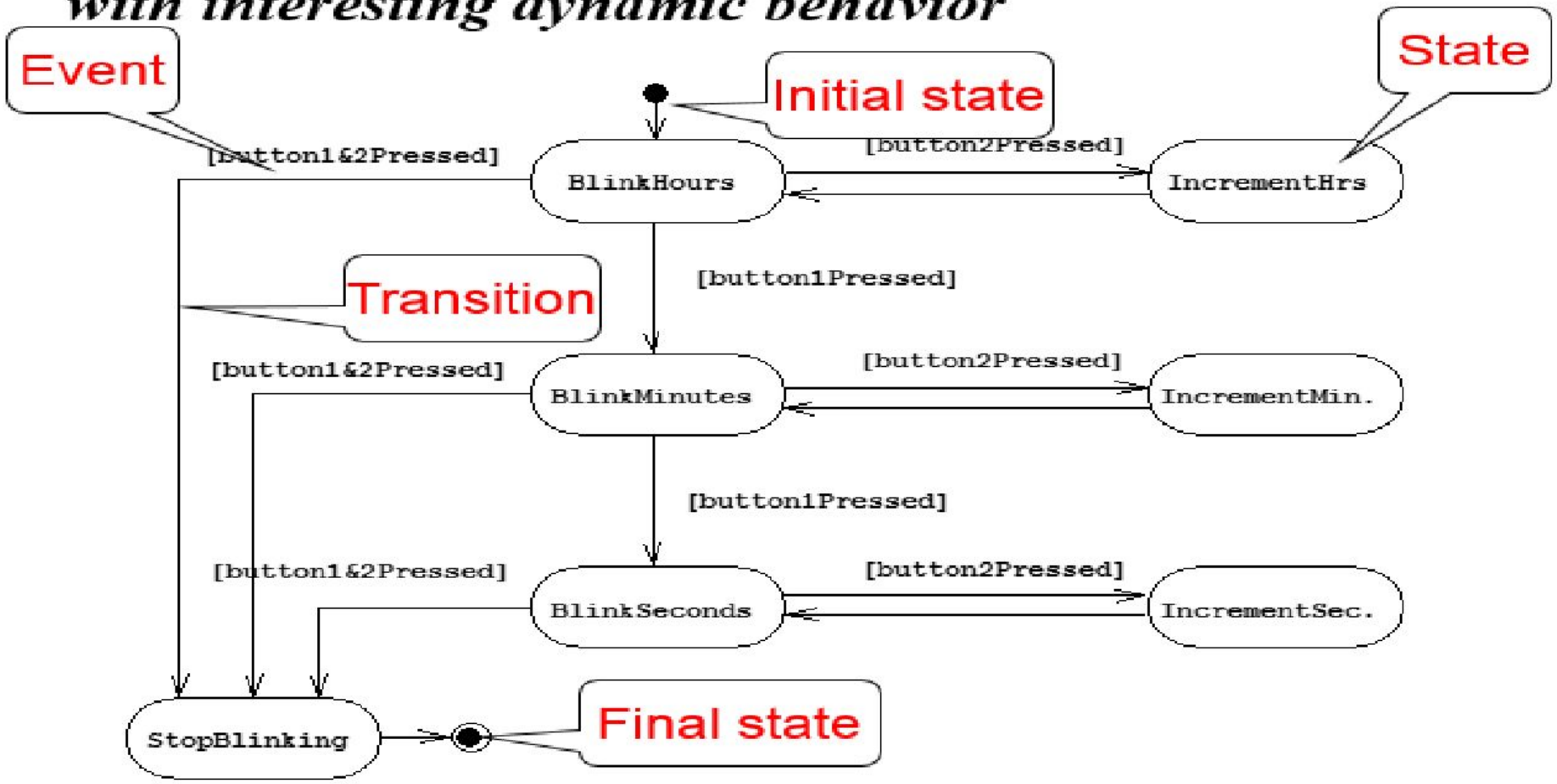


UML first pass: Sequence diagram



Sequence diagrams represent the behavior as interactions

UML first pass: Statechart diagrams for objects with interesting dynamic behavior



Represent behavior as states and transitions